

RELATÓRIO DE DESENVOLVIMENTO TÉCNICO

SISTEMA DE PLANEJAMENTO DE VIAGENS INTERATIVO

Nome: Áthur da Costa Machado

Instituição: Instituto Federal do Rio Grande do Sul (IFRS) - Campus Sertão

Curso: Análise e Desenvolvimento de Sistemas

1. Introdução

O desenvolvimento da aplicação "Xirlei, Planejador de Viagens", uma solução de software baseada na biblioteca React.js e integrada ao ecossistema de ferramentas modernas de build através do Vite. O objetivo principal do projeto é oferecer uma plataforma robusta e intuitiva para o gerenciamento de itinerários, orçamentos e cronogramas de viagens. A interface conta com um design responsivo baseado na biblioteca Bootstrap, garantindo uma experiência de usuário fluida, com suporte nativo a temas modernos (Dark Mode) e persistência de dados local segura.

2. Arquitetura de Componentes e Responsabilidades

A arquitetura da aplicação foi concebida seguindo rigorosamente os padrões de componentização e o princípio de responsabilidade única. O fluxo de dados opera de forma centralizada a partir do componente raiz, permitindo que subcomponentes especializados gerenciem fatias isoladas da interface do usuário.

A tabela a seguir descreve a divisão modular dos componentes e suas respectivas atribuições lógicas dentro do ecossistema do sistema:

Componente	Tipo de Arquivo	Responsabilidade Principal
App	.jsx (Raiz)	Gerenciamento do estado global das viagens, sincronização com o armazenamento local (localStorage) e centralização das funções CRUD (Create, Read, Update, Delete).
Viagem	.jsx	Renderização da listagem de destinos cadastrados, controle do estado de alternância para o modo de edição em tempo real e intermediação de ações de exclusão e navegação de rotas.
AdicionarViagem	.jsx	Formulário estruturado de entrada de dados para novas viagens, realizando validações de campos obrigatórios e conversões de tipos primitivos (ex: strings para formato numérico).
EditarViagem	.jsx	Formulário em linha (inline) carregado dinamicamente para a modificação dos dados de um destino específico, capturando alterações de texto, valores monetários e cronogramas antes do salvamento.
Detalhes	.jsx	Exibição detalhada de um destino selecionado, recuperando informações dinamicamente a partir dos parâmetros de busca injetados diretamente na URL da aplicação.

Componente	Tipo de Arquivo	Responsabilidade Principal
DeletarViagem	.jsx	Componente de alerta e formulário em linha (inline) para confirmação de exclusão. Substitui a linha do destino ativo para prevenir remoções acidentais, aprimorando a segurança e a fluidez da Experiência do Usuário

3. Gerenciamento de Estado e Ciclo de Vida

O ciclo de vida e a reatividade do sistema são orquestrados por hooks nativos do React. Para mitigar o problema da perda de dados decorrente da atualização da página, implementou-se uma estratégia de sincronização assíncrona com a API de armazenamento web local.

- **useState com Inicialização Preguiçosa (Lazy Initialization):** O estado inicial da lista de viagens é recuperado através de uma função de callback que lê o banco de dados do navegador apenas uma vez durante a montagem inicial do componente raiz, prevenindo gargalos desnecessários de processamento e leitura de disco.
- **useEffect para Persistência:** Um observador reativo monitora continuamente o estado da coleção de viagens. Sempre que uma inserção, remoção ou modificação ocorre, os dados estruturados em formato JSON são automaticamente serializados e gravados na chave local especificada.

4. Roteamento e Manipulação de Parâmetros de URL

A navegação entre as telas é realizada por meio da biblioteca React Router DOM. Para transferir os dados contextuais de uma viagem para a tela de visualização detalhada de forma performática e sem a dependência de estados globais complexos, utilizou-se o recurso de

parâmetros de busca nativos da URL (Query Parameters).

Através do objeto utilitário `URLSearchParams`, strings de texto complexas contendo títulos, descrições estruturadas e moedas formatadas são codificadas com segurança e injetadas na rota de destino. Ao carregar o componente filho, as informações são extraídas em tempo de execução pelos hooks de leitura de parâmetros, reconstruindo perfeitamente o contexto original da consulta do usuário.